Tokenization and Embeddings

Subword Tokens, Vector Spaces, and How Word and Sentence Embeddings Are Trained

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

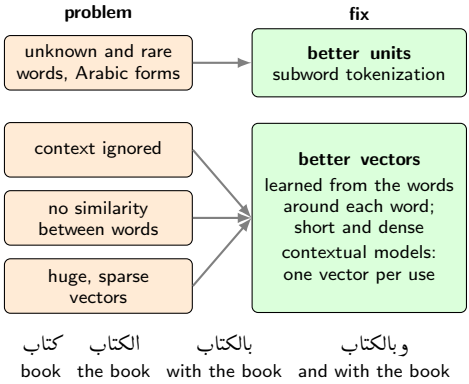
November 9, 2026

Table of Contents

1. Motivation: where counting words breaks; two fixes, better units and better vectors
2. Levels of tokenization: words, characters and subwords, and the vocabulary-size trade-off
3. Subword tokenizers: byte-pair encoding worked by hand, WordPiece, Unigram, and pitfalls
4. Tokens across languages: fertility, the Arabic token premium, and what it costs
5. From tokens to vectors: one-hot vectors, a co-occurrence matrix built by hand, Euclidean distance and cosine similarity
6. Word embeddings: short, dense, learned vectors; analogies as vector arithmetic
7. How word embeddings are trained: CBOW and skip-gram, word2vec in practice, choosing between word2vec, GloVe and fastText, evaluation
8. Contextual and sentence embeddings: one vector per use of a word, one vector per text, semantic search
9. Choosing an embedding model: benchmarks, what to check, Arabic and cross-lingual search
10. Summary and references

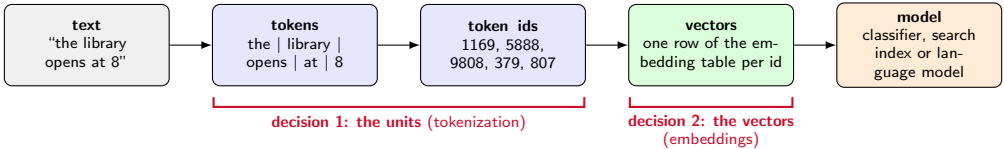
Where Counting Words Breaks

- Bag-of-words vectors and n-gram models give every whole word its own column. Four problems follow.
- **Unknown and rare words.** In the 25,000 IMDb training reviews, 30,052 of the 74,849 distinct words (40%) occur in a single review; a word never seen in training is dropped or becomes <UNK>. Arabic clitics multiply the forms (below the diagram).
- **No context.** A word's column is the same wherever it appears: *bank* by a river and *bank* that holds money are one column. Bag of words ignores the neighbouring words; an n-gram model sees only the last $N-1$.
- **No similarity.** *great* and *excellent* share no column, so what a model learns about one tells it nothing about the other.
- **Size.** 44,797 columns for IMDb, of which a review uses 136 on average.



IMDb: scikit-learn `CountVectorizer`, lower-cased, words in at least two reviews; computed for this course.

Two Decisions, in This Order



Ids from the GPT-2 tokenizer; every tokenizer has its own vocabulary and assigns its own ids.

- Every modern NLP system, from a sentiment classifier to a chat model, makes the same two decisions before it learns anything: **what the units are**, and **what vector each unit gets**.
- The units come first because the embedding table has one row per token in the tokenizer's vocabulary: the tokenizer decides which rows exist, training decides what they contain. A model's embeddings work only with the tokenizer it was trained with.

Tokens Are the Unit of Cost and Context

- The choice of unit also has a price. A language model reads and writes tokens; its **context window** (how much text it can see at once) is a number of tokens.
- Hosted models are priced per token, and generation time grows with the number of tokens produced.
- So the same document can be cheap or expensive, and fit in the window or not, depending only on how the tokenizer splits it.
- Petrov et al. (2023) found that the tokenizer used by ChatGPT and GPT-4 needs about **3 times** as many tokens for Arabic as for the same text in English, and up to 15 times for some languages.

One sentence, two languages, three tokenizers

"The students went to the library to study artificial intelligence"

ذهبت الطالبات إلى المكتبة لدراسة الذكاء الاصطناعي

Tokenizer	English	Arabic
GPT-2	10	47
Qwen3.5	10	15
AraBERT v0.2	26	7

Token counts measured for this course with the Hugging Face tokenizers of each model.

Petrov et al., "Language Model Tokenizers Introduce Unfairness Between Languages," NeurIPS 2023, arXiv:2305.15425v2, Section 1.

Learning Outcomes

By the end of this lecture you should be able to:

- **Compare** word, character and subword tokenization, and **run** byte-pair encoding merges by hand.
- **Measure** tokenizer fertility and **estimate** how tokenization changes the cost and context use of Arabic text.
- **Build** word vectors from co-occurrence counts and **compare** them with Euclidean distance and cosine similarity.
- **Train** word2vec on a corpus, **set** its main parameters, and **choose** between word2vec, GloVe and fastText.
- **Choose** an embedding model for Arabic and English, and **use** it to search documents by meaning.
- **Prerequisite knowledge:** bag-of-words and TF-IDF vectors; Zipf's law; neural networks, softmax, cross-entropy and gradient descent; logistic regression.

Three Levels of Tokenization

- The first decision is the unit. Whole words fail on unknown and rare words; three levels, on one phrase:

Unit	"unbelievably long words"	Vocabulary	Main problem
Word	unbelievably | long | words	100,000s; still incomplete	unseen words become one unknown token
Character	u | n | b | e | l | . . .	tens to thousands	sequences several times longer; each unit means little
Subword	un | ##bel | ##ie | ##va | ##bly | long | words	30,000 to 250,000	splits depend on the training corpus

- **Subwords** keep frequent words whole and break rare words into frequent pieces, so every word can be written with a vocabulary of fixed size; every large language model in use today tokenizes this way. The subword row is the actual output of the English BERT tokenizer, where "##" marks a piece that continues a word.

- Word-level vocabularies leak: the GloVe vocabulary of about 400,000 words, built from text collected around 2014, has no entry for *tokenizer*, *covid* or *chatgpt*.

Tokens from `google-bert/bert-base-uncased`; vocabulary lookups in the 6B-token GloVe release; checked for this course.

Vocabulary Size Is a Trade-Off

- A **larger vocabulary** gives shorter sequences (fewer tokens to process and pay for, more text per context window) and more whole words.
- It also costs parameters: the embedding table has one row of d numbers per token, so $V \times d$ weights, and rare tokens get few training updates.
- A **smaller vocabulary** keeps the table small but makes every text longer.
- Recent multilingual models use far larger vocabularies (XLM-R: 250,002 tokens; Qwen3.5: 248,077) so that many scripts get whole-word tokens.

Model	V	d	$V \times d$
BERT-base	30,522	768	23.4 M
GPT-2 (small)	50,257	768	38.6 M
Qwen3-0.6B	151,936	1,024	155.6 M
Qwen3.5-4B	248,320	2,560	635.7 M

V: rows of the embedding table, read from each model's configuration file (it can be slightly larger than the tokenizer's vocabulary); d: embedding width. Qwen3-0.6B has 751.6 M weights in total, so about a fifth of them sit in its embedding table.

Byte-Pair Encoding: Learning the Merges

1. Split every word of the training corpus into characters, with an end-of-word marker `</w>`.
2. Count every pair of adjacent symbols, weighted by word frequency.
3. Merge the most frequent pair into one new symbol and add it to the vocabulary.
4. Repeat until the vocabulary reaches the target size.

Toy corpus from the BPE paper:
low $\times 5$, *lower* $\times 2$, *newest* $\times 6$, *widest* $\times 3$.

Sennrich et al., "Neural Machine Translation of Rare Words with Subword Units," ACL 2016, arXiv:1508.07909v5, Algorithm 1.

Step	Merge	Pair count
1	e + s → es	9
2	es + t → est	9
3	est + </w> → est</w>	9
4	l + o → lo	7
5	lo + w → low	7
6	n + e → ne	6
7	ne + w → new	6
8	new + est</w> → newest</w>	6
9	low + </w> → low</w>	5
10	w + i → wi	3

The paper's Algorithm 1, run for this course. "es" wins first: it occurs in *newest* (6) and *widest* (3).

Applying the Merges to New Words

- To tokenize a word, start from its characters and apply the learned merges **in the order they were learned**.
- Words never seen in training still get a segmentation, built from pieces that were frequent.
- The pieces often line up with morphemes (*low*, *est*), because frequent letter sequences tend to be meaningful parts of words.
- Real tokenizers learn tens of thousands of merges on billions of words, so most common words end up as a single token.

Unseen word	Tokens after the 10 merges
lowest	low | est</w>
slowest	s | low | est</w>
newer	new | e | r | </w>
wider	wi | d | e | r | </w>

Computed for this course with the ten merges learned from the toy corpus. With more merges, *er* would become one piece too.

The Subword Family

Method	How the vocabulary is built	Used by
BPE	merge the most frequent adjacent pair, repeatedly	GPT family, Qwen, many LLMs
WordPiece	choose pieces so that the training corpus, once segmented, needs as few pieces as possible; "##" marks a continuation	BERT (about 30,000 pieces)
Unigram LM	start from a large candidate set and remove pieces that least reduce the likelihood of the corpus under a unigram language model; a word can have several segmentations	XLM-R (through SentencePiece)
SentencePiece	a library that trains BPE or Unigram directly on raw sentences, treating the space as an ordinary symbol ("_")	T5, XLM-R and many multilingual models

- All four learn the vocabulary from data, so the same word is split differently by different models. A tokenizer always ships with its model and cannot be swapped without retraining.
- GPT-style tokenizers run BPE on the raw bytes of the text, so no input is ever unknown. A script the tokenizer rarely saw in training falls back to single bytes, sometimes half a letter each.

Wu et al., 2016, arXiv:1609.08144v2; Kudo, ACL 2018, arXiv:1804.10959v1; Kudo and Richardson, EMNLP 2018 Demos, arXiv:1808.06226v1; Radford et al. (GPT-2), OpenAI, 2019, Section 2.2.

One Arabic Sentence, Four Tokenizers

ذهبت الطالبات إلى المكتبة لدراسة الذكاء الاصطناعي ("The students went to the library to study artificial intelligence")

Tokenizer	Tokens	Pieces (decoded, in reading order)
AraBERT v0.2	7	one token per word
Qwen3.5	15	ذه | بت | الطالب | ات | إلى | المكت | بة | ل | در | اسة | ...
XLM-R	17	ذهب | ت | الطالب | ات | إلى | المكتب | ة | لدراسة | ...
GPT-2	47	mostly single letters, many split into raw bytes that are not valid characters on their own

- A tokenizer trained with plenty of Arabic finds clitics and stems (*al-*, *-aat*); one trained mostly on English falls back to letters and bytes.
- AraBERT pays the opposite price: the English version of this sentence costs it 26 tokens, against 10 for GPT-2.

Tokenizations computed for this course with the Hugging Face tokenizers of each model.

Tokenizer Pitfalls

Input	GPT-2	Qwen3
2026	20|26	2|0|2|6
1234567	123|45|67	1|2|3|4|5|6|7
hello	id 31373	id 14990
␣hello	id 23748	id 23811
HELLO	HE|LL|O	HEL|LO

Token splits and ids from the Hugging Face tokenizers of `gpt2` and `Qwen3-0.6B`; ␣ marks a leading space. Computed for this course.

- `bert-base-uncased` lower-cases first, so *hello*, *Hello* and *HELLO* all become id 7592.

- **Numbers:** GPT-2 cuts digits into chunks that depend on the training data, so digit positions are inconsistent. Qwen3 splits every number into single digits.
- **Spaces and case:** *hello* with and without a leading space, or in capitals, are unrelated ids. The model has to learn that they mean the same.
- **Glitch tokens:** the tokenizer is trained before the model, so some vocabulary entries almost never appear in the model's training data and their embeddings are barely trained. Such under-trained tokens were found across many models, and can induce unwanted model behaviour.

Land and Bartolo, "Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models," EMNLP 2024, arXiv:2405.05417v2.

Fertility: Tokens per Word

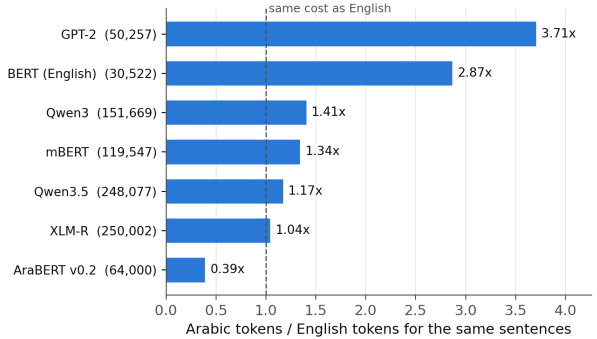
- **Fertility** is the average number of tokens a tokenizer produces per word. A fertility of 1 means every word is in the vocabulary.
- Rust et al. (2021) found that multilingual BERT over-segments Arabic, and that giving a model a tokenizer built for the language improved its results on almost every task.
- English fertility is close to 1.3 for every general tokenizer here. Arabic fertility ranges from 1.3 to 5.7.

Tokenizer	Vocabulary	English	Arabic
GPT-2	50,257	1.27	5.71
BERT (English)	30,522	1.30	4.52
mBERT	119,547	1.38	2.26
Qwen3	151,669	1.30	2.22
Qwen3.5	248,077	1.29	1.84
XLM-R	250,002	1.41	1.78
AraBERT v0.2	64,000	2.76	1.31

Tokens per whitespace word on 913 English–Arabic sentence pairs (8 to 40 English words) from the OPUS-100 test split; measured for this course.

Rust et al., "How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models," ACL 2021, arXiv:2012.15613v2.

The Arabic Token Premium, Measured



Tokens for the 913 Arabic sentences divided by tokens for their English translations; tokenizer vocabulary in brackets. Measured for this course on OPUS-100.

- Fertility per word hides that Arabic packs clitics into fewer, longer words. The fair comparison is tokens for the **same content**.
- English-centred tokenizers charge Arabic about 3 to 4 times as much, in line with the factor of about 3 that Petrov et al. report for the GPT-4 tokenizer.
- Large multilingual vocabularies bring the premium close to parity; an Arabic-only tokenizer reverses it.

Zhang et al., "Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation" (OPUS-100), ACL 2020, arXiv:2004.11867v1.

What the Premium Costs

Content that takes **10,000 tokens** in English, written in Arabic:

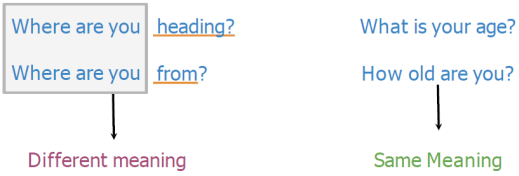
Tokenizer	Arabic tokens
GPT-2	37,100
Qwen3	14,100
Qwen3.5	11,700
AraBERT v0.2	3,900

10,000 $\times$ the measured Arabic/English ratio, rounded to hundreds.

- **Money:** per-token pricing scales the bill by the same factor.
- **Context:** a fixed window holds proportionally less Arabic text, so fewer retrieved passages fit next to a question.
- **Time:** generation produces one token per step, so an Arabic answer takes longer to write.
- **Quality:** pieces that split letters carry little meaning, and the model has seen each of them in fewer contexts.

In practice: before adopting a model for Arabic, tokenize a sample of your own documents and measure the ratio. It takes a few lines of code and changes cost estimates by a factor of up to four.

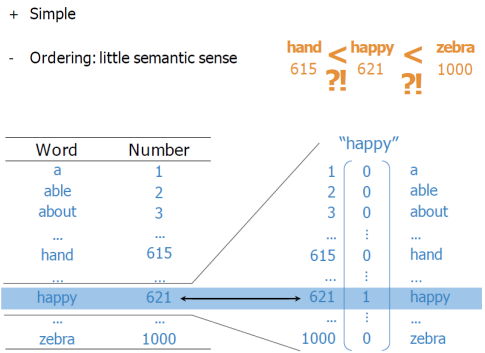
Why Represent Meaning as Vectors?



DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

- The second decision: the vector. Tokenization ends with an integer id for every token; each id needs a vector, and the vectors should reflect meaning.
- Left pair: the same first three words, different meanings. Right pair: no words in common beyond *you*, same meaning. String overlap gets both pairs wrong.
- A **vector space model** puts each word or text at a point, so that texts with similar meaning are near each other and "are these similar?" becomes a distance or an angle.
- Uses: search and question answering (find the passage nearest a question), paraphrase and duplicate detection, clustering, and features for classifiers.

The Simplest Word Vector: One-Hot



DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

- The integer id itself is a poor vector: its order means nothing. *hand* $= 615 <$ *happy* $= 621 <$ *zebra* $= 1000$ says nothing about the words.
- A **one-hot** vector for id i has V entries, all 0 except a 1 at position i. No order is implied.
- One-hot vectors are the building block of bag of words: add up the one-hot vectors of a document's tokens and the result is its count vector.
- They are huge, since V runs from tens of thousands to a million, and all but one entry is 0.

One-Hot Vectors Know Nothing About Meaning

- Two different one-hot vectors never have a 1 in the same place, so their dot product is 0: *hotel* is as unrelated to *motel* as to *banana*.
- Every word is equally far from every other. A sentiment model that learns from "the hotel was dirty" learns nothing about "the motel was dirty".
- A one-hot vector says *which* word this is, never what it means. The meaning has to come from somewhere else: the text around the word.

	hotel	*motel*	*banana*	*the*	*was*	*dirty*
hotel	1	0	0	0	0	0
motel	0	1	0	0	0	0
banana	0	0	1	0	0	0

$hotel \cdot motel = 0$ $\quad$ $motel \cdot banana = 0$

$hotel \cdot banana = 0$

One-hot vectors over a vocabulary of $V = 6$ words. Every pair of different words gives the same dot product, 0.

You Shall Know a Word by the Company It Keeps

"You shall know a word by the company it keeps"
Firth, 1957





(Firth, J. R. 1957:11)

J. R. Firth, *A Synopsis of Linguistic Theory, 1930–1955*, 1957.

Figure: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

- The **distributional hypothesis**: words that appear in similar contexts tend to have similar meanings. A word's neighbours supply what its one-hot vector lacks.
- Meaning can then be measured from raw text, without labels, by counting or predicting which words occur near which.

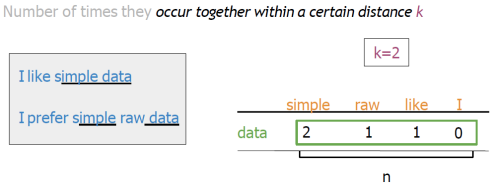
a cup of **tea** with milk
she ordered **tea** in the morning

a cup of **coffee** with milk
she ordered **coffee** in the morning

the **engine** needs new oil
the **engine** stalled on the road

Shared contexts pull *tea* and *coffee* together; *engine* keeps different company. The training signal is free, which is why word vectors can be learned from billions of words.

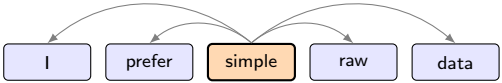
Building a Co-occurrence Matrix by Hand



Two sentences, window $k = 2$, and the finished row of *data*.

DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

1. List the vocabulary: *I*, *like*, *prefer*, *simple*, *raw*, *data*. Start from a 6×6 table of zeros, one row and one column per word.
2. Choose a window: here $k = 2$ words on each side of a word.
3. Visit every token. For each other token at most k positions away, add 1 to the cell (token, neighbour).
4. When every token has been visited, each row is that word's vector: how often each vocabulary word appeared near it.



Centre word *simple* in "I prefer simple raw data": all four other words are within 2 positions, so add 1 to (*simple*, *I*), (*simple*, *prefer*), (*simple*, *raw*), (*simple*, *data*).

The Finished Co-occurrence Matrix

	I	*like*	*prefer*	*simple*	*raw*	*data*
I	0	1	1	2	0	0
like	1	0	0	1	0	1
prefer	1	0	0	1	1	0
simple	2	1	1	0	1	2
raw	0	0	1	1	0	1
data	0	1	0	2	1	0

Window $k = 2$ over "I like simple data" and "I prefer simple raw data". Computed for this course.

- Row *data* $= (0, 1, 0, 2, 1, 0)$: the counts of the figure, in the column order used here.
- The table is symmetric: *data* near *simple* is the same event as *simple* near *data*.
- *like* and *prefer* never occur within two words of each other (their cell is 0), yet both rows have a 1 under *I* and under *simple*: they keep the same company, which one-hot vectors could not show.
- On real text the matrix is $V \times V$, one row and one column per word, and mostly zeros, like the bag-of-words matrix.

Word-by-Document Design

Number of times a word *occurs within a certain category*

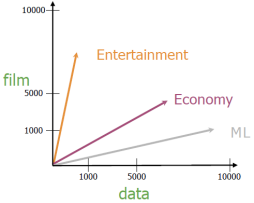
Corpus

	Entertainment	Economy	Machine Learning
data	500	6620	9320
film	7000	4000	1000

DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

- The window can be as large as a whole document. Here each column counts a word's occurrences in one **category** of documents: entertainment, economy or machine learning.
- Read by **rows**, the matrix gives word vectors: $\textit{film} \mapsto (7000, 4000, 1000)$.
- Read by **columns**, it gives document vectors: the economy corpus $\mapsto (6620, 4000)$ over *data* and *film*. With one column per document, this is the bag-of-words document-term matrix turned on its side.
- Word-by-word suits word similarity; word-by-document suits classifying and searching documents.

Documents as Points: Distance or Angle?



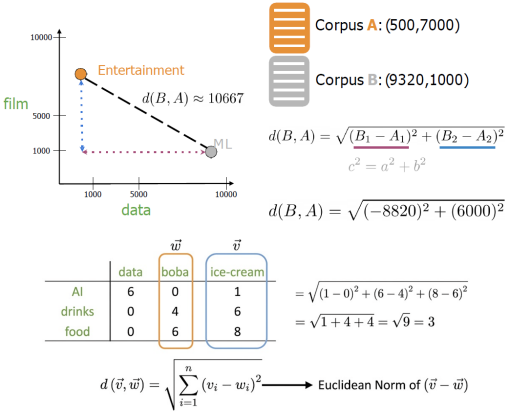
	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of "similarity:"
Angle
Distance

DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

- Both designs give every word or document a vector of counts. Here each corpus is plotted by its counts of *data* and *film*. Economy and machine learning point in similar directions; entertainment points elsewhere.
- Two ways to turn "close" into a number: the **distance** between the points, or the **angle** between the arrows.
- Real vectors have thousands of dimensions, but both measures are computed the same way as in two.

Euclidean Distance



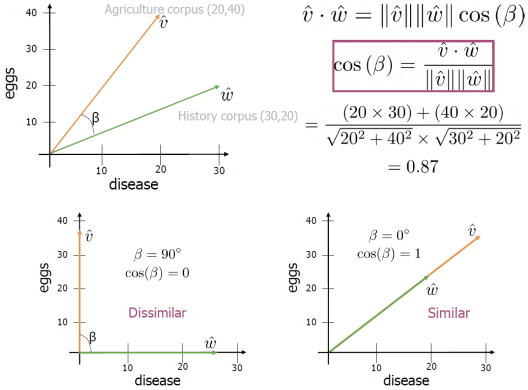
DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

- The straight-line distance between two points, from Pythagoras:

$$d(\mathbf{v}, \mathbf{w}) = \sqrt{\sum_{i=1}^{n} (v_i - w_i)^2} = \|\mathbf{v} - \mathbf{w}\|$$

- Top: corpora A $(500, 7000)$ and B $(9320, 1000)$ are $\sqrt{8820^2 + 6000^2} \approx 10{,}667$ apart.
- Bottom: the same formula for word vectors with three entries: *boba* and *ice-cream* are $\sqrt{1+4+4} = 3$ apart.
- In NumPy: `np.linalg.norm(v - w)`.

Cosine Similarity



DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

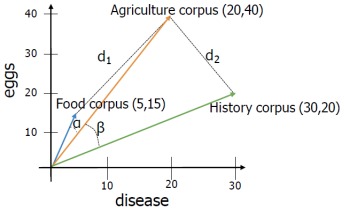
- The cosine of the angle β between the vectors:

$$\cos\beta = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \, \|\mathbf{w}\|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2} \sqrt{\sum_i w_i^2}}$$

- Top: the agriculture $(20, 40)$ and history $(30, 20)$ corpora give $\frac{20 \cdot 30 + 40 \cdot 20}{\sqrt{20^2+40^2}\sqrt{30^2+20^2}} = 0.87$.
- Bottom: perpendicular vectors give $\cos 90° = 0$ (nothing in common); parallel ones give $\cos 0° = 1$. With counts, which are never negative, the cosine stays between 0 and 1; learned vectors can reach -1.

Euclidean Distance or Cosine Similarity?

- Cosine similarity when corpora are different sizes



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle between the vectors

DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3.

- The food corpus is small; agriculture and history are large. By distance, agriculture is closer to history ($d_2 = 22.4$) than to food ($d_1 = 29.2$): size dominates.
- By angle, agriculture is closer to food ($\alpha < \beta$; cosine 0.99 against 0.87), because both mention *eggs* far more often than *disease*.
- Cosine ignores length, so a short and a long document on the same topic still match. That is why text search uses cosine, or the dot product of vectors scaled to length 1, which gives the same ranking.

Cosine Between Two Co-occurrence Rows, by Hand

	I	*like*	*prefer*	*simple*	*raw*	*data*
like	1	0	0	1	0	1
prefer	1	0	0	1	1	0
product	**1**	0	0	**1**	0	0

dot product $1 + 1 = 2$

length of each row $\sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}$

cosine $\dfrac{2}{\sqrt{3}\sqrt{3}} = \dfrac{2}{3} = 0.67$

Pair	Shared neighbours	Cosine
like, *prefer*	*I*, *simple*	0.67
like, *data*	*simple*	0.47
I, *data*	*like*, *simple*	0.83

- Two rows of the hand-built matrix. Multiply matching entries and add them up (the dot product), then divide by the two lengths.
- Only the columns where **both** words have a neighbour add to the score: *I* and *simple*.
- *like* and *prefer* never occur near each other, yet score 0.67: words are compared through the neighbours they share.
- Two sentences are too few to trust. *I* and *data* come out most similar, only because both happen to sit next to *like* and *simple*. Useful rows need millions of words of text.

Nearest Words in 25,000 Movie Reviews

- The same recipe at scale: (1) build the co-occurrence matrix of the IMDb training reviews, with $k = 2$; (2) for a query word, compute the cosine between its row and every other word's row; (3) list the words with the highest cosine.

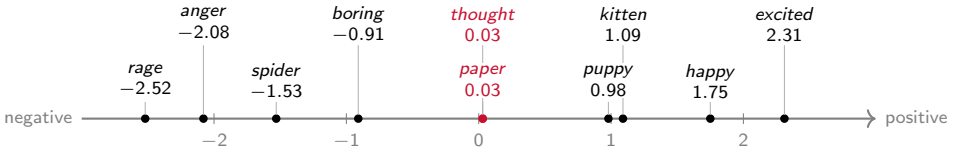
Query	Four nearest words, with cosine similarity			
great	wonderful 0.96	brilliant 0.95	good 0.94	fine 0.93
boring	dull 0.97	confusing 0.96	silly 0.96	predictable 0.95
music	dialogue 0.98	animation 0.98	dialog 0.98	cinematography 0.98
awful	terrible 0.94	awesome 0.93	horrible 0.93	amazing 0.92
actor	actress 0.96	artist 0.92	alien 0.88	assassin 0.88

- **It works without labels:** *great* lands next to *wonderful*, *boring* next to *dull*, *music* next to other parts of a film.
- **Same neighbours is not same meaning:** "the film was awful" and "the film was awesome" give the two words the same neighbours, so their rows almost match. *actor* sits with other words that follow "an".
- **Frequent words dominate:** the commonest neighbours of *great* are *the*, *is* and *and*, so every row looks alike and every cosine above lies between 0.87 and 0.98.

13,316 words seen at least 20 times in 5.6 million lower-cased tokens. Computed for this course.

Describing a Word with a Few Numbers

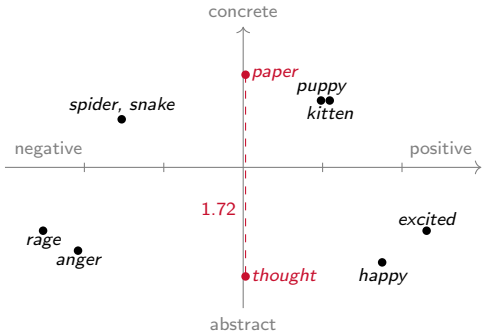
- A co-occurrence row describes a word with one count per vocabulary word: 13,316 numbers for the IMDb matrix, most of them about *the*, *is* and *and*.
- Another way: give the word a few numbers, each one a score for a **property**. Start with one property, how negative or positive the word is:



- Each word is now a single number, and words that feel alike get nearby numbers: *happy* and *excited*, *rage* and *anger*.
- One property is not enough. *paper* and *thought* both score 0.03 and look identical, although one is an object and the other an idea.

Values from DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4; redrawn for this course.

A Second Axis: Each Word Becomes a Point



Horizontal: the same sentiment scores. Vertical: abstract ($-$) to concrete ($+$). Values from DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4; redrawn for this course.

- Add a second property, from abstract to concrete. Each word now has two numbers and is a point: $paper = (0.03, 0.79)$, $thought = (0.03, -0.93)$.
- *paper* and *thought* are now 1.72 apart, while *puppy* and *kitten* stay close (0.11 apart).
- That list of numbers is the word's **vector**: Euclidean distance and cosine similarity apply to it unchanged.
- Learned embeddings use hundreds of such axes (300 for the GloVe vectors used here). Nobody chooses them: training places the words so that words used in similar contexts end up close, and single axes rarely have a clean name like "sentiment".

From One-Hot to Dense: the Embedding Matrix

- An **embedding** gives each word a dense vector of d learned numbers (d is typically 100 to a few thousand) in place of its V-entry one-hot vector or co-occurrence row.
- Stack the vectors into a $V \times d$ **embedding matrix** E. Multiplying the one-hot vector of word i by E picks out row i, so in code it is a table lookup: `torch.nn.Embedding(V, d)`.
- The goal of training: words used in similar contexts get nearby rows.

one-hot ($V = 6$)

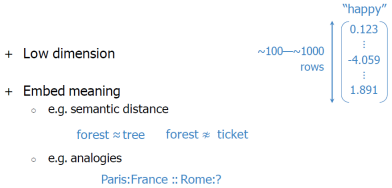
hotel	`[1 0 0 0 0 0]`
motel	`[0 1 0 0 0 0]`
banana	`[0 0 1 0 0 0]`

dense ($d = 3$, illustrative)

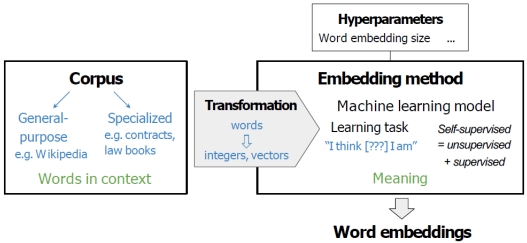
hotel	`[0.81  0.10  0.52]`
motel	`[0.77  0.18  0.49]`
banana	`[-0.20  0.93  0.05]`

With these dense vectors, $\cos(\textit{hotel}, \textit{motel}) = 0.996$ and $\cos(\textit{hotel}, \textit{banana}) = -0.05$; with one-hot vectors both are 0. The three dense vectors are made up to show the idea.

Word Embeddings: Short, Dense and Learned



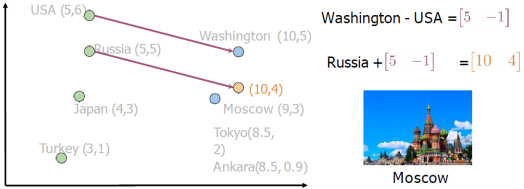
- A few hundred numbers per word instead of V, and distances that mean something: *forest* close to *tree*, far from *ticket*.



- Three ingredients: a **corpus** (general, like Wikipedia, or specialised, like contracts), a **learning task** built from it, and **hyperparameters** such as the vector size.
- The corpus decides what "similar" means: legal text gives legal neighbours.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4.

Vector Arithmetic Captures Relations



DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3; the analogy result is due to Mikolov et al. (2013).

- Toy 2-D vectors: Washington − USA $= (5, -1)$. Adding it to Russia $(5, 5)$ gives $(10, 4)$, and the nearest word is Moscow $(9, 3)$.

Query	Nearest word	Cosine
king − man + woman	queen	0.67
paris − france + italy	rome	0.74
riyadh − saudi + egypt	cairo	0.74
walking − walk + swim	swimming	0.68

Real 300-dimensional GloVe vectors (6B-token release, about 400,000 words), with the three query words excluded from the answers; computed for this course.

- Directions in the space carry relations (capital-of, gender, tense), learned from co-occurrence alone: nobody labelled capitals.

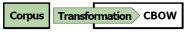
Turn Raw Text into a Prediction Task

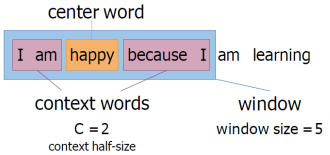


The little ____?____ is barking



dog
puppy
hound
terrier
...



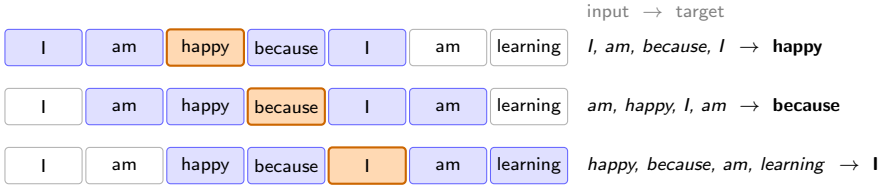


- To learn such vectors from raw text, hide a word and predict it from its neighbours. Words that can fill the same gap (*dog*, *puppy*, *hound*) must end up with similar vectors to make similar predictions.
- Every position in the corpus gives one training example: a **centre word** and its **context words**, the C words on each side (here $C = 2$, a window of 5).
- The labels come from the text itself, so this is **self-supervised** learning: any amount of raw text is training data.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4.

From a Sentence to Training Examples

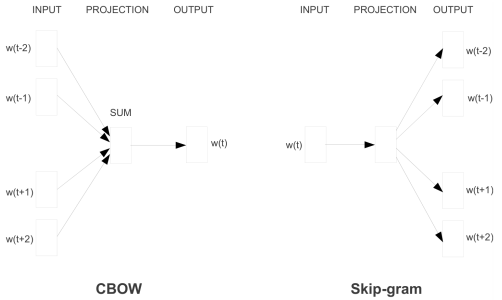
- Slide a window along "I am happy because I am learning", one word at a time, with $C = 2$ words on each side:



Orange: the centre word. Blue: its context words, the training input.

- Every position gives one training example: the context words go in, and the model must name the centre word. Here are three of the seven positions; at the edges of the sentence the window is shorter.
- The text supplies both the input and the answer, so nobody labels anything: a corpus of a billion words gives about a billion examples.

word2vec: CBOW and Skip-gram



Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," ICLR Workshop 2013, arXiv:1301.3781v3, Figure 1.

- **CBOW** (continuous bag of words) predicts the centre word $w(t)$ from its context, combined in a single projection layer.
- **Skip-gram** does the reverse: it predicts each context word from the centre word. In "the cat **sat** on the mat" with $C = 2$, it gets the pairs (sat, the), (sat, cat), (sat, on), (sat, the).
- Both are shallow networks whose first layer is a table with one row of numbers per word. After training, the predictions are thrown away and that table is kept: it is the word embeddings.

Training word2vec in Practice

```python
from gensim.models import Word2Vec

# one list of tokens per sentence
sents = [["i", "am", "happy", ...],
         ...]

model = Word2Vec(
    sents, vector_size=200, window=5,
    min_count=5, sg=1, negative=5,
    sample=1e-4, epochs=5, workers=16)

model.wv["king"]   # its 200 numbers
model.wv.most_similar("king")
```

Argument	Typical	What it changes
`vector_size`	100–300	how much each vector can hold; more needs more text
`window`	5–10	how far "context" reaches
`min_count`	5	rarer words get no vector
`sg`	0 or 1	0: CBOW, fast; 1: skip-gram, better for rare words
`negative`	5–20	random words per real pair
`sample`	10^{-5}–10^{-3}	how hard very frequent words are thinned out
`epochs`	5	passes over the corpus

- **Negative sampling** makes training fast: for each real (word, neighbour) pair the model only tells the real neighbour apart from a few random words, instead of scoring the whole vocabulary.
- `sample` drops many occurrences of words such as *the*, which say little about their neighbours: faster training, better vectors.

Mikolov et al., NeurIPS 2013, arXiv:1310.4546v1; word2vec project page, code.google.com/archive/p/word2vec; gensim library.

Training word2vec Ourselves

	CBOW	Skip-gram
Training time	39 s	73 s
Analogy accuracy	31.6%	37.7%
semantic	22.6%	31.0%
syntactic	36.1%	41.2%

text8: the first 10^8 characters of a cleaned English Wikipedia dump, 17.0 million words. gensim with the settings of the code above (CBOW: sg=0), 16 CPU cores; 71,290 words kept. Analogy questions from Mikolov et al. whose words are among the 30,000 most frequent. Computed for this course.

Word	Nearest neighbours (CBOW)
king	prince, throne, kings, vii, reigning, queen
france	spain, italy, belgium, portugal, netherlands
computer	computers, computing, console, hardware
coffee	cocoa, sugar, tea, wool, wheat
doctor	detective, cleese, potter, villain, nurse

- A minute on a CPU already groups countries, computing terms and foods.
- The corpus decides what "similar" means: in Wikipedia, *doctor* keeps company with fictional characters such as *Doctor Who*.

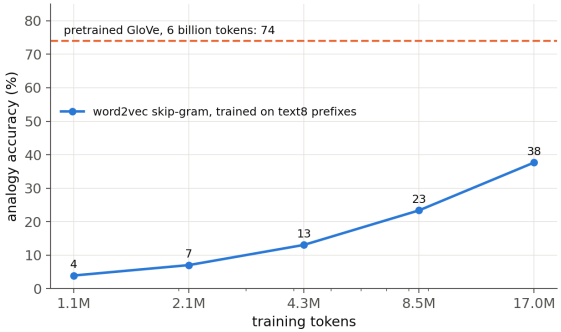
word2vec, GloVe or fastText?

	word2vec	GloVe	fastText
Learns from	predicting words in sliding windows	the co-occurrence matrix, counted once	sliding windows, plus character pieces
Unseen word	no vector	no vector	built from its pieces
Ready-made	Google News: 3M words, 300 dims	Wikipedia + Gigaword: 400K words	157 languages, Arabic included, 300 dims
Load	gensim: `word2vec-google-news-300`	gensim: `glove-wiki-gigaword-300`	fastText: `cc.ar.300.bin`
Use it when	you train on your own corpus	you want ready-made English vectors	Arabic, rich morphology, typos, rare words

- **GloVe** fits the vectors so that the dot product of two words matches the log of how often they occur together: the co-occurrence matrix, compressed into short dense rows.
- **fastText** splits *playing* into <`pl`, `pla`, `lay`, `ayi`, `yin`, `ing`, `ng`> plus the whole word, and sums their vectors: an unseen or misspelled word still gets a vector.

Pennington et al., EMNLP 2014, D14-1162; Bojanowski et al., TACL 2017, arXiv:1607.04606v2; Grave et al., LREC 2018, arXiv:1802.06893v2; gensim-data and fastText download lists.

More Text, Better Vectors



Skip-gram trained on the first 1/16, 1/8, 1/4, 1/2 and all of text8, same settings; GloVe 300d (6B tokens) scored on the same questions. Computed for this course.

- Analogy accuracy rises from 4% to 38% as the corpus grows from 1 to 17 million words, and has not levelled off.
- Vectors trained on 6 billion tokens reach 74%: corpus size is the strongest lever.
- This is why one usually downloads pretrained vectors, and trains from scratch only when no pretrained model covers the language or domain and there is plenty of raw text.

Evaluating Word Embeddings

Intrinsic: test the vectors directly

- **Analogies:** Mikolov et al.'s set has 8,869 semantic questions (capital of, currency of) and 10,675 syntactic ones (plural, past tense). A question counts as correct only if the nearest word is exactly right.
- **Word similarity:** correlate the cosine of word pairs with human similarity ratings.
- Fast and cheap, but a high score does not guarantee a better downstream system.

Extrinsic: test them inside a task

- Plug the vectors into the system you care about (a classifier, a search engine) and measure that system's metric.
- Slower, but it answers the real question.
- The same split applies to the sentence embeddings below: benchmark scores first, then your own questions and documents.

Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," ICLR Workshop 2013, arXiv:1301.3781v3, Section 4.1.

The Limits of One Vector per Word

- **One vector for all senses.** The nearest GloVe neighbours of *bank* are all financial; the river bank is averaged away.
- **A frozen vocabulary.** Words coined after the corpus was collected have no vector at all (fastText softens this).
- **Learned bias.** Vectors absorb associations in the training text. Bolukbasi et al. showed word2vec vectors trained on news completing "man is to computer programmer as woman is to" with *homemaker*.
- **Word vectors, not text vectors.** Averaging them for a sentence loses word order and negation.

GloVe neighbours of bank	
banks	0.70
banking	0.60
central	0.54
credit	0.53
bankers	0.52
financial	0.50

Cosine similarity, 300-dimensional GloVe; computed for this course.

Bolukbasi et al., "Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings," NeurIPS 2016, arXiv:1607.06520v1.

Contextual Embeddings: One Vector per Use of a Word

- ELMo (2018) and BERT (2019) are deep networks pretrained on large corpora to predict words from their context. Each outputs one vector per token, computed from the **whole sentence**.
- The same word in different sentences gets different vectors, so senses separate.
- In the table, the vector for *bank* in two river sentences is far more similar than across a river and a money sentence.
- For this lecture the network is a black box: a sentence goes in, one vector per token comes out.

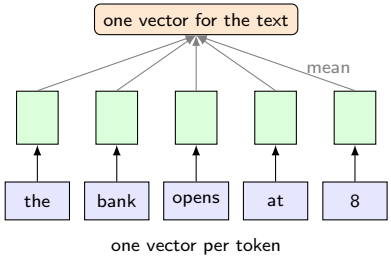
Sentence pair (vector of bank)	Cosine
river: "we sat on the bank of the river and watched the water" / "the boat drifted toward the muddy bank of the stream"	**0.85**
money: "she deposited the cheque at the bank on her way to work" / "the bank approved the loan for the new office"	**0.77**
river vs money (first sentence of each)	0.50
river vs money (second sentence of each)	0.38

Last-layer vectors of `bert-base-uncased`; computed for this course.

Peters et al., "Deep contextualized word representations," NAACL 2018, arXiv:1802.05365v2; Devlin et al., "BERT," NAACL 2019, arXiv:1810.04805v2.

From Token Vectors to One Vector per Text

- Search, clustering and duplicate detection need **one vector per text**, so that two texts can be compared with a single cosine.
- The simplest way: average the token vectors (**mean pooling**).
- A network trained only to predict words gives poor averages: Reimers and Gurevych found averaged BERT vectors worse than averaged GloVe vectors at judging sentence similarity.
- **Sentence-embedding models** fix this. They are trained on pairs of texts that belong together (a question and its answer, a title and its abstract), over a billion pairs for the models below, so that texts with the same meaning get nearby vectors. A text goes in; one vector comes out.



Reimers and Gurevych, "Sentence-BERT," EMNLP 2019, arXiv:1908.10084v1, Table 1; model cards of `all-MiniLM-L6-v2` (over 1 billion pairs) and `multilingual-e5-small` (1 billion multilingual pairs, Wang et al., 2024).

Sentence Embeddings in Code

```
from sentence_transformers import \
    SentenceTransformer

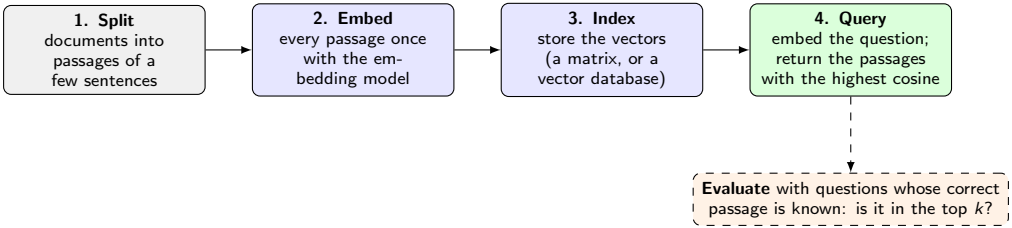
model = SentenceTransformer(
    "all-MiniLM-L6-v2")
texts = ["The pump is overheating.",
    "The motor is running too hot."]
a, b = model.encode(texts)
model.similarity(a, b)       # 0.64
```

Compared with "The pump is overheating."	MiniLM	E5-small
The motor is running too hot.	0.64	0.91
Close the valve before maintenance.	0.37	0.83
درجة حرارة المضخة مرتفعة جدًا (Arabic: the pump's temperature is very high)	0.13	0.83

Cosine similarity; each E5 input starts with `query:`. `all-MiniLM-L6-v2` and `multilingual-e5-small`, both 384 numbers per text. Computed for this course.

- **Compare scores within one model only.** E5 puts almost every pair between 0.7 and 1.0, as its model card says, while MiniLM spreads them out. What matters is the order.
- The English-only MiniLM gives the Arabic translation 0.13. E5 scores it only barely above a different instruction (0.829 against 0.827), yet over 500 translation pairs it finds the right one 89.8% of the time. **Judge a model on a test set of your own questions**, never on a few examples.

Semantic Search in Four Steps



- Steps 1 to 3 run once per document collection; step 4 runs for every question in milliseconds.
- The same pipeline, with the passages handed to a language model to write an answer, is the retrieval half of question answering over private documents.
- Keep the TF-IDF baseline next to it: keyword search still wins on exact names, codes and numbers.

Benchmarks for Embedding Models: MTEB and MMTEB

MTEB (2023)

- 8 task types (retrieval, clustering, classification, re-ranking, semantic similarity, ...), 58 datasets, 112 languages, with a public leaderboard.
- Main finding: no single embedding method was best on every task type.

MMTEB (2025)

- A community extension with over 500 tasks in more than 250 languages.
- Models with billions of parameters led some languages and task types, but the best public model overall was `multilingual-e5-large-instruct`, with 560 million parameters.

Reading a leaderboard

- Filter to the task type you need (retrieval for search) and the languages you need (Arabic).
- An average over all tasks can hide a weak score on yours.
- Public test sets leak into training data; the final check is always your own documents and questions.

Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," EACL 2023, arXiv:2210.07316v3; Enevoldsen et al., "MMTEB: Massive Multilingual Text Embedding Benchmark," ICLR 2025, arXiv:2502.13595v4.

What to Check Before Choosing a Model

Property	Why it matters
Languages	an English-only model cannot match an Arabic question to an English passage, or even place Arabic sentences sensibly
Maximum input length	passages longer than the limit (often 512 tokens; up to 8,192 for some models) are cut off silently
Vector size	a million passages at 1,024 dimensions in 32-bit floats take about 4 GB of memory
Model size	sets encoding speed, and whether it runs on a CPU or needs a GPU
Input format	some models expect prefixes such as `query:` and `passage:`; leaving them out lowers quality
Licence	whether the weights may be used commercially and on premises

- Then measure: 30 to 50 real questions with known answer passages tell you more than any leaderboard average.

Cross-Lingual Search, Measured

Task: for each of 500 Arabic sentences, find its English translation among the 500 English sentences, by cosine similarity alone.

Model	Params	Dim.	Top-1
all-MiniLM-L6-v2	23 M	384	2.0%
multilingual-e5-small	118 M	384	**89.8%**

all-MiniLM-L6-v2 is trained on English only. OPUS-100 English–Arabic test pairs; random guessing scores 0.2%. Measured for this course.

- Same vector size, same cosine search; only the training data differs.
- The multilingual model was trained on text pairs in many languages, so a sentence and its translation land close together.
- This is what lets an Arabic question retrieve an English policy document: **cross-lingual retrieval**.

Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," 2024, arXiv:2402.05672v1.

Candidate Models for Arabic and English

Model	Languages	Notes
all-MiniLM-L6-v2	English	23 M parameters, 384 dimensions; fast on a CPU; a good English baseline
multilingual-e5 (small, base, large)	100	trained on 1 billion multilingual text pairs; needs `query:` / `passage:` prefixes
multilingual-e5-large-instruct	100	560 M parameters; best public model on MMTEB's multilingual benchmark
BGE-M3	100+	inputs up to 8,192 tokens; one model gives dense, sparse (keyword-like) and multi-vector representations

- For Arabic search, start from a multilingual model, check that Arabic is in its training languages, and test dialect queries as well as MSA ones.
- Training languages matter more than size: in the cross-lingual test, an English model with the same vector size found 2% of the translations.

Chen et al., "M3-Embedding" (BGE-M3), 2024, arXiv:2402.03216v5; Wang et al., Multilingual E5, 2024, arXiv:2402.05672v1; model card of `all-MiniLM-L6-v2`, huggingface.co/sentence-transformers.

Summary

- **Tokens are the unit of cost and context.** Subwords keep frequent words whole and spell rare ones from pieces. BPE merges the most frequent pair, again and again; tokenizers that work on bytes never meet an unknown input. Numbers, spaces and case are split in ways the model must learn around.
- **Arabic pays a token premium in English-centred tokenizers:** 3.7 times for GPT-2 on the same sentences, close to 1 for large multilingual vocabularies.
- **Vector spaces turn similarity into geometry.** One-hot vectors share nothing; co-occurrence counts compare words by their company. Cosine ignores length, Euclidean distance does not; analogies are vector arithmetic.
- **Word embeddings are learned by prediction.** CBOW predicts the centre word from its window, skip-gram the reverse; the network's first table becomes the embeddings. In gensim it is one call. GloVe learns from the co-occurrence matrix; fastText builds vectors from character pieces, so unseen and Arabic word forms still get one.
- **Data size dominates:** our skip-gram rose from 4% to 38% analogy accuracy between 1 and 17 million words; GloVe on 6 billion tokens reaches 74%.
- **Static vectors merge senses.** Contextual models give one vector per use of a word; sentence-embedding models, trained on over a billion text pairs, give one vector per text, so one cosine compares two texts.
- **Semantic search in four steps:** split the documents, embed every passage once, index the vectors, and return the passages nearest the question. Score it with questions whose passage is known, and keep TF-IDF next to it for names and codes.
- **Choose, then measure.** Shortlist with MTEB, check languages and input length, then score on your own questions: on Arabic–English translation pairs an English-only model finds 2% of the matches, multilingual-e5-small 89.8%.

References

1. Petrov, La Malfa, Torr and Bibi, "Language Model Tokenizers Introduce Unfairness Between Languages," NeurIPS 2023. arXiv:2305.15425v2
2. Sennrich, Haddow and Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL 2016. arXiv:1508.07909v5 (Algorithm 1)
3. Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," 2016. arXiv:1609.08144v2
4. Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates," ACL 2018. arXiv:1804.10959v1
5. Kudo and Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing," EMNLP 2018 Demos. arXiv:1808.06226v1
6. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019. cdn.openai.com
7. Land and Bartolo, "Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models," EMNLP 2024. arXiv:2405.05417v2
8. Rust et al., "How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models," ACL 2021. arXiv:2012.15613v2
9. Zhang et al., "Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation" (OPUS-100), ACL 2020. arXiv:2004.11867v1
10. Firth, "A Synopsis of Linguistic Theory, 1930–1955," in *Studies in Linguistic Analysis*, Blackwell, 1957.

References (cont.)

11. DeepLearning.AI, Natural Language Processing Specialization: Course 1, *NLP with Classification and Vector Spaces*, Week 3; Course 2, *NLP with Probabilistic Models*, Week 4 (source of the figures so credited). deeplearning.ai

12. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," ICLR Workshop 2013. arXiv:1301.3781v3 (source of Figure 1)

13. Mikolov et al., "Distributed Representations of Words and Phrases and their Compositionality," NeurIPS 2013. arXiv:1310.4546v1

14. word2vec project page (training settings and pretrained Google News vectors). code.google.com/archive/p/word2vec

15. Mahoney, "About the Test Data" (the text8 corpus), 2006. mattmahoney.net/dc/textdata.html

16. Pennington, Socher and Manning, "GloVe: Global Vectors for Word Representation," EMNLP 2014. aclanthology.org/D14-1162

17. Bojanowski et al., "Enriching Word Vectors with Subword Information," TACL 2017. arXiv:1607.04606v2

18. Grave, Bojanowski, Gupta, Joulin and Mikolov, "Learning Word Vectors for 157 Languages," LREC 2018. arXiv:1802.06893v2

19. Bolukbasi et al., "Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings," NeurIPS 2016. arXiv:1607.06520v1

20. Peters et al., "Deep contextualized word representations," NAACL 2018. arXiv:1802.05365v2

21. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL 2019. arXiv:1810.04805v2

References (cont.)

22. Reimers and Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP 2019. arXiv:1908.10084v1 (source of Figures 1 and 2)
23. Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," 2024. arXiv:2402.05672v1
24. Model card of `intfloat/multilingual-e5-small`. huggingface.co/intfloat/multilingual-e5-small
25. Model card of `sentence-transformers/all-MiniLM-L6-v2`. huggingface.co/sentence-transformers/all-MiniLM-L6-v2
26. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," EACL 2023. arXiv:2210.07316v3
27. Enevoldsen et al., "MMTEB: Massive Multilingual Text Embedding Benchmark," ICLR 2025. arXiv:2502.13595v4
28. Chen et al., "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024. arXiv:2402.03216v5
29. Antoun, Baly and Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," OSACT 2020. arXiv:2003.00104v4